Optimization of Empty Container Repositioning by Integrating Large Language Models and Multi-Agent Systems

Kangzhen Peng[1], Shida Xu[1], Yuhong Wang[1], Zhenkai Wang[1], Mengying Hua[1], Zhihong Jin[1,]

[a]*College of Transportation Engineering, Dalian Maritime University, No. 1 Linghai Road, Dalian, 116026, Liaoning, China*
[b]*College of Economics and Management, Dalian Jiaotong University, , Dalian, 116028, Liaoning, China*
[c]*Faculty of Maritime and Transportation, Ningbo University, No. 169, Qixing South Road, Ningbo, 315211, Ningbo, China*
[d]*ENN Group, Hongrun Road, Langfang, 065001, Heibei, China*
[e]*Sinotrans Limited, No. 85 Renmin Road, Dalian, 116001, Liaoning, China*

Abstract

The problem of empty container repositioning is a core challenge within the container supply chain domain. This study addresses the limitations of existing solvers that depend on manual modeling and problem-specific customization by designing a multi-agent collaborative framework grounded in artificial intelligence large language models. This framework interprets the intricate business scenarios associated with the empty container repositioning problem through natural language instructions and employs the OR-Tools solver to achieve end-to-end optimization. To validate the efficacy of the proposed method, benchmark tests were conducted utilizing the ComplexOR and NLP4LP datasets. The findings exhibit that our approach enhances the accuracy rate to 86.59% in comparison to traditional baseline methods. Implementing this method for an empirical analysis within the Liaoning coastal port cluster-Northeast hinterland, the system optimized repositioning routes and decreased total costs by 12.18%. This verifies its practical applicability in facilitating intelligent decision-making for the shipping industry. The research outcomes have the potential to offer shipping companies scalable

*Corresponding author. Tel.: +86; fax: +86.

intelligent decision support.

Keywords:
large language model decision, multi-agent system, end-to-end optimization, mixed-integer programming, container supply chain, empty container repositioning

---

1. Introduction

Mixed-Integer Programming (MIP) constitutes a fundamental instrument within the domain of Container Supply Chain Management (SCM), extensively employed in essential scenarios such as resource allocation, route planning, and scheduling decisions. However, its practical implementation is heavily dependent on domain experts who manually translate intricate business challenges into variables, objectives, and constraints, a procedure that is not only arduous and time-consuming but also results in inflexible models that inadequately respond to dynamically evolving business requirements. Despite the fact that algebraic modeling languages (e.g., AMPL, GAMS) have marginally enhanced modeling efficiency, they remain inadequate in realizing end-to-end optimization from unstructured business descriptions to MIP formulation.

Recent advancements in Large Language Models (LLMs), particularly in the realms of natural language understanding and code generation, present transformative potential for the automation the modeling process. These models' substantial in-context learning and natural language comprehension capabilities allow them to discern business intents, such as "minimize total cost" and initially generate corresponding mathematical formulations or solution code, thereby substantially reducing the complexity associated with modeling. Nevertheless, the inherent "hallucination" issue of LLMs and their lack of reliability in executing complex multi-step logical reasoning render them challenging to trust directly in operations research (OR) tasks that demand mathematical precision. As probabilistic generative models, LLMs might produce output that appear plausible but are mathematically unsound, which considerably restricts their practical application in critical decision-making systems.

To address these challenges, the integration of the reasoning capabilities of LLMs with the collaborative control advantages of Multi-Agent Systems (MAS) represents a promising technical pathway. MAS proficiently han-

dle complexity by means of task decomposition and distributed collaboration. Therefore, the principal scientific inquiry of this paper is: How can a novel LLM-driven multi-agent collaborative paradigm be constructed that not only comprehends practical business intents but also, through rigorous workflow orchestration and internal verification mechanisms, guarantees reliability, controllability, and interpretability throughout the entire transition from natural language to final solution. Addressing this issue will establish a general framework for achieving automated and adaptive large-scale complex optimization.

Empty Container Repositioning (ECR) is one of the most representative and challenging problems in container SCM, encompassing complex multi-period, multi-node, and multi-modal transportation decisions, wherein conventional MIP modeling approaches encounter substantial obstacles. Consequently, this paper employs the ECR problem as an empirical case study to validate the effectiveness and superiority of the proposed paradigm. This not only contributes to addressing a specific industry challenge but also robustly demonstrates the framework's strong potential in tackling complex real world issues.

The contributions of this paper are fourfold: 1) The introduction of workflow orchestration within LLM-driven multi-agent collaboration, shifting from static code generation to dynamic, adaptive task planning, thereby overcoming the rigid collaboration rules characteristic of traditional MAS; 2) The design of a systematic set of constraints and verification mechanisms to mitigate LLM hallucinations, thereby providing a novel technical path for the reliable application of LLMs in precision-critical computing domains through the integration of "multi-agent division of labor, standardized operating procedures, and anomaly backtracking mechanisms"; 3) The implementation of an end-to-end automated modeling and solving framework that translates ECR natural language descriptions of business optimization problems into business solutions, which significantly lowers the application barrier for MIP; 4) The exploratory application of LLM and MAS in container supply chain optimization, evidenced by the successful application of the proposed paradigm to the practical industry challenge of sea-land coordinated ECR faced by shipping companies, thereby not only validating the method's effectiveness but also presenting a novel solution for the digital and intelligent transformation of the port and shipping industry.

2. Literature Review

This research focuses on a category of problems that are essential and extensively applied within the field of OR: the automated modeling and resolution of MIP problems. This section provides a review of pertinent advancements that are central to this core theme, establishing the foundation for the subsequently proposed paradigm, which employs container SCM as a representative application scenario.

2.1. Traditional MIP Solution

The resolution of MIP problems generally adheres to a systematic OR framework that encompasses five stages. Initially, a mathematical model is constructed to abstract the real-world issue, Subsequently, an analysis of the problem's properties is undertaken to identify an appropriate solution strategy. The third stage involves the design and evaluation of algorithm performance. The fourth phase encompasses programming tasks to invoke optimization solvers and the execution of numerical simulations. Finally, the effectiveness of the solution is assessed, followed by its implementation. A core bottleneck within this framework lies in the significant dependence on experts for the laborious and time-intensive responsibilities of model development and parameter tuning.

While MIP exhibits theoretically and algorithmically mature, its practical application encounters fundamental challenges. The modeling phase, serving as the cornerstone of the entire process, continues to present considerable difficulties. This phase necessitates experts to convert complex real-world scenarios into mathematical variables, objective functions, and constraints, which is not only resource-intensive and protracted but also significantly reliant on expert experience [**?**]. This dependency constitutes the fundamental impediment to its extensive and rapid adaptation to novel scenarios. Models developed for specific context often require expert re-evaluation and modification when business conditions alter (e.g., incorporating new warehouse capacity constraints), which suggests a limited capacity for generalization. As real-world issues increase exponentially in scale and complexity, the traditional method reliant on individual experts for modeling proves inadequate to satisfy these demands.

In response to the challenging nature of employing general-purpose programming languages for complex MIP models, Algebraic Modeling Languages (AMLs) have emerged. Prominent examples, such as AMPL and

GAMS, aim to elevate the level of abstraction from procedural code to algebraic expressions, thereby decoupling models from data and enhancing modeling efficiency. In recent developments, optimization libraries grounded in general-purpose programming languages (e.g., Pyomo, PuLP) are embedded as libraries within modern high-level languages, facilitating the use of robust data science ecosystems and toolchains for the seamless integration of optimization models and solvers.

Nonetheless, AMLs remain unable to automatically transform unstructured business descriptions into MIP formulations. For instance, within the context of container supply chain optimization, changes in the business environment necessitate experts to manually reinterpret new requirements, such as converting the need to "avoid empty container accumulation at ports" into balance constraints involving inventory variables. This demonstrates the significant reliance of AMLs on expert modeling. Additionally, the specific syntax of AMLs acts as an impediment for domain experts without technical backgrounds (e.g., shipping company schedulers), thus restricting the broad adoption of optimization technologies.

2.2. Emerging Paradigms from LLMs in Scientific Computing and OR

The emergence of LLMs in recent years brings revolutionary opportunities to the domains of scientific computing and OR. These models, trained on extensive corpora, demonstrate powerful capabilities in natural language understanding, code generation, and mathematical reasoning. LLMs offer an innovative approach to address the previously mentioned challenges. Their exceptional performance in open-domain natural language understanding significantly enhances efficacy in tasks involving arithmetic, commonsense, and symbolic reasoning. Chain-of-Thought (CoT) prompting, initially proposed by [**?**], enables models to deconstruct complex problems into multiple steps, substantially improving LLMs' reasoning ability without the need for fine-tuning, thus allowing them to tackle mathematical word problems and commonsense reasoning effectively. The introduction of CoT suggests that LLMs are also capable of resolving complex mathematical problems requiring precise, step-by-step calculation. Moreover, CoT contributes to the interpretability and reliability of LLMs. Traditionally, large-scale unsupervised deep learning models have been perceived as opaque entities with enigmatic decision-making processes, which raises concerns about the credibility of their results. By decomposing logical reasoning problems into sequentially solvable steps, CoT provides a transparent logical framework for the results

generated, thus offering a level of explicability. Research based on CoT has expanded considerably. The conceptualizations of Tree of Thoughts [**?**] and Graph of Thoughts [**?**] have further enhanced reasoning capability by allowing LLMs to explore and integrate thoughts in a structured manner. These studies demonstrate the potential of LLMs in resolving complex problems, especially those involving reasoning and mathematics.

Concurrently, the utilization of LLMs in scientific computing is transitioning from proof-of-concept to practical application, showing growing potential in disciplines such as computer science, transportation, logistics, and SCM. For example, in the domain of complex system simulation, [**?**] developed a natural language interface employing adaptive dialog agents that comprehend user intent and dynamically generate task flows, thereby demonstrating the capacity of LLMs to simplify the complexity associated with simulation modeling. In the specific context of OR, the natural language processing capability of LLMs offers a novel avenue to overcome the modeling bottleneck associated with MIP. Taking the representative MIP problem of vehicle routing as an example, [**?**] noted that LLMs are capable of precisely translating business descriptions such as "minimize total transportation cost," "ensure inventory balance across warehouses," or "meet customer time window constraints" into structured decision variables, objective functions, and constraints. This capacity provides strong support for rapid prototyping and model iteration, thereby significantly enhancing modeling efficiency. [**?**] employed LLMs for traffic flow prediction, while [**?**] concentrated on code generation. For modeling new scenarios, LLMs have the capability to automatically generate complete optimization models and solving code on the basis of scenario descriptions provided by algorithm engineers, in consequence significantly reducing development timelines.

In spite of their considerable potential, LLMs operating as solitary agents encounter following challenges when handling complex optimization tasks: 1) Hallucination and Uncontrollability: LLMs are inherently probabilistic generative models, and as such, they may produce code that appears plausible but is mathematically invalid, leading to challenges in ensuring the accuracy and controllability of results [**?**]. 2) Deficiency in Transparency and Interpretability: Numerous LLMs function as "black box" models, rendering it challenging to understand their decision-making processes and to verify results in critical decision-making scenarios. 3) Constraints in Multi-step Task Handling: The computational and decision-making demands placed on an individual LLM can increase dramatically when engaging with MIP tasks

that necessitate complex, multi-step reasoning and planning [**?**].

2.3. Distributed Collaboration and Optimization in MAS

In response to the constraints associated with LLMs, researchers have commenced investigations into technical pathways that integrate LLMs with MAS. The latter can effectively handle complex problems by leveraging multiple autonomous, interacting agents working collaboratively. Research focusing on LLM-based multi-agent collaboration has revealed that dynamic interaction, characterized by both cooperation and competition, frequently lead to unanticipated efficacy. [**?**] proposed the AutoGen framework to facilitate the implementation of LLM applications through the configuration of multiple agents. Empirical studies have demonstrated the framework's effectiveness across diverse applications, such as mathematical problem-solving/coding, conversational question answering, operational optimization, and online decision-making. [**?**] introduced Standard Operating Procedures (SOP) into MAS to facilitate software development. Within the domain of operations optimization, existing studies have engaged LLMs in addressing problems such as linear programming and MIP. For example, [**?**] attempted to resolve the Traveling Salesperson Problem utilizing LLM based reasoning, demonstrating commendable performance on small-scale instances. [**?**] introduced the NL4Opt competition, which aimed at the extraction of meaning and formulation of optimization problems derived from natural language, involving subtasks of identifying semantic labeling and generating logical representations. The competition has the potentially to enhance the interactive capability between non-expert users and optimization solvers. [**?**] designed a system intended for formulating and resolving (Mixed Integer) Linear Programming problems based on natural language descriptions. This system is proficient of developing mathematical models and writing debugging solver code. [**?**] devised a multi-expert collaboration framework, known as Chain-of-Experts (CoE), aimed at automating the modeling and programming process for complex OR problems, ultimately diminishing dependency on domain-specific experts. Experimental results showed that CoE significantly outperformed state-of-the-art LLM methods in terms of performance on both LPWP and ComplexOR benchmarks.

2.4. Overview of Empty Container Repositioning Challenges

The issue of ECR presents a significant challenge within the domain of container supply chain, and has long been an important research topic in

academia. Traditionally, ECR research has concentrated on single mode of repositioning between seaports via maritime routes, emphasizing the optimization of components such as ship routing [? ? ?], shipping network design [? ? ?], and slot allocation [?] through MIP. [?] explored the dynamics of pricing competition in maritime transport under blockchain technology and ECR, determining perceived container leasing prices through inverse optimization methodologies. These investigations tend to disregard the particularities associated with land transport – specifically, when shipping companies utilize their own fleets, the marginal cost of sea repositioning approaches to near zero, starkly contrasting the costly nature of land transport. Inland container transportation, which encompasses the terrestrial component of the transport chain, significantly drives costs in container shipping, accounting for 50%-70% of total container transport volume and potentially up to 40% of total transport costs, thus constituting a critical bottleneck in the pursuit of operational optimization within the shipping sector. Research on inland ECR has gradually increased in recent years. [?] studied a hinterland transport network consisting of a seaport and a dry port, devising an ECR model that accounts for both laden and empty container transport to minimize costs, showing that the utilization of dry ports can reduce laden container transport costs. [?] investigated ECR patterns between port nodes and inland depot nodes within their respective zones, providing insights applicable to practical ECR challenges. As inland dry ports evolve, they, alongside seaports, form intermodal systems facilitated by railway collaboration; however, these entities frequently belong to disparate organizations (such as railway companies and shipping companies), which function with independent decision-making processes and competitive interactions. [?] and [?] addressed ECR issues from the perspective of sea-rail intermodal transport, by establishing multimodal container repositioning systems with nodes including dry port and seaport. They analyzed benefit distribution between railway companies operating at dry ports and liner shipping companies operating at seaports, under both centralized and decentralized decision-making systems, through numerical exemplifications. The extant body of research predominantly operates within a framework governed by either seaports or dry ports, managing repositioning initiatives via either seaports or dry ports, and addressing sea and land nodes in isolation. In reality, within the hinterland collection and distribution network, dry ports serve as extensions of seaports, and their collaboration can effectively diminish the detention rates of empty container.

In summary, the key to addressing the above issues lies in deeply integrating the strengths of LLMs and MAS to construct a new paradigm driven by LLMs and powered by collaborative MAS. In this paradigm, the LLM acts as a high-level planner, receiving natural language descriptions of business problems, leveraging its capabilities for abstraction and planning to automatically generate structured mathematical models, as well as MAS task allocation and collaboration rules, thereby fundamentally overcoming the cognitive dependency bottleneck inherent in traditional modeling. While recent works such as OptiMUS [**?**] and CoE [**?**] have demonstrated promising results in automating the formulation of linear and MIP from natural language, they primarily focus on syntactic correctness and benchmark performance on standardized datasets (e.g., NLP4LP, ComplexOR). In contrast, our work advances beyond proof-of-concept automation toward industrial-grade deployability in a high-stakes vertical domain, maritime logistics.

1) **Problem Context:** Whereas existing methods target generic or synthetic optimization problems, we address ECR, a real-world, multi-node, multi-modal operational challenge characterized by dynamic imbalances, contractual constraints, and significant cost implications.

2) **Workflow Architecture:** OptiMUS employs a single LLM with iterative refinement; CoE uses a fixed pipeline of expert agents. Our SOP-MAS framework introduces dynamic workflow orchestration, where agent activation is adaptively determined by problem state and verification outcomes, enabling context-aware collaboration rather than rigid sequencing.

3) **Verification and Human Integration:** Prior approaches rely heavily on solver feedback or automated unit tests. We embed a dual-layer validation mechanism: (i) programmatic constraint checking by the Testing Engineer Agent, and (ii) structured human-in-the-loop approval from domain experts, with explicit logging and backtracking support. This ensures not only mathematical feasibility but also business acceptability, a critical requirement absent in most academic benchmarks.

Thus, building upon recent advances in LLM-based optimization, our framework represents a concrete step toward industrial deployment by integrating

dynamic multi-agent collaboration, auditable verification, and deep vertical specialization within the context of maritime container logistics.

3. Methodology

This research proposes a specialized LLM-based MAS framework predicated upon the Standardized Operating Procedure (SOP) paradigm. Drawing inspiration from the work of [**?**], which incorporated SOP into MAS for the purpose of software development, this study adapts and extends the concept to the domain of ECR optimization. The foundation of this framework is centered around a novel workflow orchestration and a carefully designed set of agents tailored for ECR optimization tasks.

3.1. Agent Definition

Given an agent set $\{a_i\}_{i=0}^{5}$, which comprises six distinct agents: a Project Manager Agent (PM-Agent), a Data Engineer Agent (DE-Agent), an SCM Modeling Expert Agent (ME-Agent), a Python Developer Agent (PY-Agent), a Testing Engineer Agent (TE-Agent), and a Maritime Business Analyst Agent (BA-Agent).

1) **PM-Agent:** Responsible for executing state transitions and managing task allocation. It initiates appropriate agents, allocates tasks in alignment with the established plan, and monitors project progress to ensure the overall task is completed smoothly and robustly.

2) **DE-Agent:** Its role is in data engineering, facilitating the transformation of raw user input (e.g. natural language requirements and data sources including but not limited to CSV/JSON) into structured modeling parameters requisite for mathematical modeling. It records all data processing activities and quality concerns, thereby ensuring the parameters are directly operable by the ME-Agent.

3) **ME-Agent:** Converts the natural language problem description into a computable mathematical model.

4) **PY-Agent:** Obtaining the optimization model from the ME-Agent and acquiring parameters from the DE-Agent. Upon this information, it composes a singular, robust, and efficient Python script, which employs an optimization solver to address the specified optimization

Figure 1: SOP-MAS Flowchart

problem. The resultant output from the PY-Agent consists of a code block that includes a comprehensive, executable function, as well as any required import statements.

5) **TE-Agent:** Receiving the Python code authored by the PY-Agent along with the data curated by the ME-Agent. It carries out the execution of the code within a sandbox environment, monitors the execution process, assesses the outcome of "optimization solution success/failure," and subsequently provides a report on the execution.

6) **BA-Agent:** The optimization within container supply chains often involves complex MIP issues, which are traditionally resolved through automatic solvers such as Gurobi, CPLEX, and OR-Tools. However, business stakeholders often encounter difficulties in comprehending the rationale underpinning the optimization outcomes, leading to repeated communication with experts and a slow decision-making process that is dependent on manual interpretation. Therefore, informed by concepts in the literature [**?**], this paper defines the BA-Agent.

Each agent, denoted as a_i, within the MAS can be formally defined as a tuple: $a_i = (f_i, L_i, \mathcal{T}_i, t_i, \mathcal{H}_i)$, where i serves as the unique identifier of the agent; f_i represents the core functional module of the agent, which processes one or more inputs to produce an output; L_i denotes the large language model selected for the agent (e.g., GPT-4, Claude 3, etc.); $\mathcal{T}_i$ refers to the specific prompt template designed for the agent's role; t_i indicates the temperature parameter utilized during LLM inference; and $\mathcal{H}_i$ represents a set of supplementary hyperparameters, such as `top_p` and `max_tokens`.

3.2. Orchestration of Workflow Based on SOP-MAS

This framework facilitates intelligent decision-making and task execution through collaborative multi-role interaction within a structured configuration. The sequence of agent interactions as outlined by the SOP is illustrated in Figure **??**, comprising the following steps:

Step 1: Parameter Setting and Initialization. Characterize the natural language description of the input problem as P, the upper limit of selection attempts as K, and the predetermined set of agents as $A = \{a_i\}_{i=0}^{n}$, where

n denotes the total number of agents. Proceed with the initialization of a message pool C, alongside the establishment of a stack H to record the selected agents, adhering to a last-in-first-out (LIFO) principle to facilitate exception backtracking.

Step 2: Agent Selection. While [**?**] also utilizes the SOP concept, it employs a sequential approach to agent selection, whereas SOP-MAS dynamically chooses agents according to sub-sequence rules. Consider the set of executable agents, excluding the PM-Agent itself, as the action space $A = \{a_i\}_{i=1}^{5}$ of the PM-Agent. Denote the output at step i as y_i, and the state as $s_i = (P, C, A, K - i)$. The policy selection function is defined as:

$$\pi_a(e \mid s) = Pr\{A_i = e \mid S_i = s\} \tag{1}$$

The strategy for agent selection can be transformed into the design of the prompt template $\mathcal{T}_i$, aiming at attaining an optimal policy through prompt engineering. The selected agent at each step is determined by $a_i = \pi_a(s_i)$.

Step 3: Action Execution. According to the agent a_i selected in the previous step, the action f_i is to be executed. The output y_i of agent a_i is obtained as $y_i = a_i(P, C)$.

Step 4: Update Shared Message Pool. The shared message pool C is implemented as a dictionary, wherein each key serves as a unique identifier, and the associated values consist of response messages generated by agents. During each inference step, the message pool is augmented by incorporating the current output $C := C \cup \{y_i\}$. The sequence of remaining selectable agents is updated in accordance with the subsequence rule: let $A_{>i} = (a_{i+1}, \ldots, a_n)$ represent the remaining agents after step i, then set $A := A_{>i}$. Push a_i onto the stack H.

Step 5: Iterative Modeling. Should the current iteration count k exceed the maximum permissible selections K, emit the message pool C and the execution result.

Step 6: Verification and Validation. If the TE-Agent is activated, it executes the code generated by the PY-Agent and captures the solver's runtime report (including status, objective value, constraint violations, and solution feasibility). A verifiable correctness gate is then applied:

- *For benchmark instances:* The solution is deemed correct only if (i) the solver status is Optimal or Feasible, and (ii) the objective value matches the ground-truth optimum.

- *For real-world ECR cases:* The solution must satisfy all constraints as programmatically verified by the TE-Agent, and pass an SCM human expert sanity check.

Only when both automated checks and expert sign-off are satisfied does the workflow advance to Step 7. Otherwise, proceed to Step 7 for error diagnosis and exception backtracking.

Step 7: Exception Backtracking. When an execution error is detected and `is_backward` = `True`, trigger the backtracking mechanism. Extract the top agent a_i from the stack H and evaluate whether its output y_i was the source of the exception. Revert the message pool C to its state prior to the execution of a_i, by removing the outputs of a_i and all subsequent agents: update $C := C \setminus \{y_i, y_{i+1}, \ldots\}$. Restore the action space A to the remaining set prior to selecting a_i. Obtain feedback error information through $(\delta_k, \xi_k) \leftarrow a_i(C)$, where δ_k is a Boolean indicating whether the exception needs to be propagated upward, and ξ_k represents a natural language error description for localization. If the result is correct, set $\delta_k :=$ False and terminate the reasoning process. Otherwise, continue propagating the exception upward. In the event that `is_backward` = `False`, proceed to Step 8.

Step 8: Termination Condition. The algorithm concludes its execution and yields the optimal solution.

It is important to clarify that the proposed SOP-MAS framework is fundamentally designed as an expert-in-the-loop decision support system, rather than a fully autonomous optimization engine. In real-world container logistics operations, particularly in complex, high-stakes scenarios like ECR, the ultimate responsibility for model validity, constraint fidelity, and operational feasibility must rest with domain experts (e.g., logistics planners, operations researchers, or port analysts).

3.3. Mechanisms for Hallucination Mitigation

Within the framework of the SOP-MAS methodology, a systematic strategy is adopted to mitigate the pervasive issue of hallucinations in LLMs. This strategy encompasses the integration of prompt engineering, SOP, and a mechanism for exception backtracking and correction.

1) **Meticulously Designed Prompt Engineering.** To ensure each agent accurately comprehends its role and tasks, and produces outputs

that are standardized, this study designs refined prompt templates for each agent. These templates adhere to a unified structure, encompassing key elements such as role definition, principal tasks, input/output specifications, and constraints. By restricting the generation scope and steering the reasoning process through prompt engineering, the LLMs are constrained to produce output strictly in JSON format or in accordance with specific code templates, thereby enforcing structured thinking and minimizing the production of redundant or irrelevant information. Owing to space limitations, this paper will not display all prompts in full but will place their core design logic along with a typical example (e.g., for the ME-Agent) in **??**.

2) **Constraining Role by the SOP.** The SOP mitigates risks by clearly defining the responsibilities of each agent and by establishing quality standards for intermediate outputs. Utilizing JSON as the communication protocol between agents inherently restricts each agent's latitude for unstructured "free-form" output, requiring their responses to adhere to specific formats and standards, thereby diminishing the likelihood of generating nonsensical or irrelevant outputs, termed hallucinations. Furthermore, the logical consistency and mathematical correctness of the generated models and code across different agents function as a mechanism for cross-verification.

3) **Exception Backtracking and Correction Mechanism.** Upon the occurrence of an exception during code execution within the sandbox environment, the TE-Agent retains the error information in the shared message pool. Concurrently, a stack pop operation is performed on H, sequentially popping the topmost agents to check whether the error was caused by the current agent's output. This backtracking mechanism allows for pinpointing the source of errors and facilitates corrective actions, significantly enhancing the robustness of the SOP-MAS framework. By enabling agents to examine and potentially amend their outputs in response to subsequent execution feedback, the system is enabled to autonomously rectify and recover from errors attributable to LLM inaccuracies.

3.4. Illustrative Example

In order to effectively demonstrate the capabilities of the proposed SOP-MAS workflow in facilitating an end-to-end optimization from natural lan-

Figure 2: SOP-MAS Flowchart

guage descriptions to formal modeling and solution, a step-by-step exposition based on a classic case of ECR in container SCM is provided, as illustrated in Figure **??**.

In this example, the user firstly inputs a natural language description P of the optimization problem, accompanied by historical data (e.g., CSV or JSON files). The PM-Agent, in accordance with Step 2, selects and assigns tasks to the appropriate agents. The DE-Agent converts unstructured data and business constraints into consistent and reproducible modeling parameters, while preserving a meticulous record of the data-processing steps. Constructed on this basis, the ME-Agent automatically formulates a multi-period, multi-node mathematical model for empty container repositioning. Subsequently, the PY-Agent generates comprehensive, executable Python code for resolving the model using the specified parameters. The TE-Agent executes the code within a sandboxed environment and returns both the solution results and a run report. Ultimately, the BA-Agent translates the model outputs—such as loading/unloading schedules, repositioning costs, and key performance indicators—into an interpretable business-analysis report for the user.

This process: 1) natural-language description, 2) structured modeling, 3) solution-code generation and execution, 4) interpretable results, largely parallels the conventional workflow of mixed-integer programming (MIP) modeling and solution. It highlights the manner in which the MAS assigns individual agents to authentic enterprise roles. Concurrently, the SOP framework bestows unique benefits in the realms of task decomposition, agent scheduling, exception backtracking, and iterative multi-round interaction.

4. System Implementation and Experimental Analysis

The SOP-MAS method is assessed against baseline methodologies utilizing publicly accessible datasets to validate the effectiveness of the algorithm. Essential parameters of the method are further analyzed through sensitivity experiments on instances of diverse scales to identify the optimal hyper-parameters combination. The entirety of the experiments is conducted in Python 3.11.9 and performed on a Windows 10 Professional platform featuring an Intel i5-4210U CPU and 32 GB of DDR3 memory.

4.1. Comparison with Baseline Methods and Effectiveness Analysis

To validate the effectiveness of SOP-MAS, a total of 269 instances were selected from the ComplexOR and NLP4LP benchmark datasets. The ComplexOR dataset [**?**] is explicitly crafted to provide benchmark tests for complex OR problems, comprising 37 multi-source, heterogeneous scenarios spanning supply chain optimization, scheduling, and warehousing logistics. Each scenario is expert-annotated, including both model formulations and test cases to verify the correctness of generated code. The NLP4LP dataset comprises LP and MIP scenarios, with problem descriptions distinguished by lengthy textual narratives, implicit constraints, and complex data structures (e.g., multi-resource allocation and production scheduling), necessitating domain expertise for accurate mathematical modeling. To strengthen evaluation fidelity, we further refined the annotation of all instances by structuring them into four standardized components:

1) a natural-language problem description (capturing business intent, constraints, and objectives),

2) structured modeling metadata (including variable definitions, parameter schemas, and constraint templates),

3) sample input data (e.g., CSV/JSON instances for model instantiation), and

4) reference optimization outcomes (optimal/near-optimal solutions and solver logs for correctness verification).

This design establishes a closed-loop evaluation framework: SOP-MAS ingests 1) to 3) and is assessed on its ability to reproduce 4) without manual intervention. This structure not only enables rigorous quantitative evaluation but also supports qualitative analysis of modeling fidelity and interpretability, key criteria for real-world deployment.

The performance of SOP-MAS on these instances is evaluated by comparing it against baseline methods on benchmark cases, specifically including the Standard Prompting Method (SPM), CoT, CoE [**?**], and OptiMUS [**?**]. We adopt DeepSeek-v3-0324 as the backbone LLM, given its leading position among open weight models. Crucially, this work does not aim to benchmark LLM capabilities; rather, it seeks to evaluate a novel workflow orchestration

Table 1: Comparison of SOP-MAS and Baseline Algorithms

Baseline	CER	RER	AR
Standard Prompting Method (SPM)	7.67%	11.81%	48.08%
Chain-of-Thought (CoT)	7.32%	10.83%	52.26%
Chain-of-Experts (CoE)	4.18%	5.77%	62.37%
OptiMUS	3.14%	4.51%	77.35%
SOP-MAS	1.39%	2.91%	86.59%

paradigm—SOP-MAS. Therefore, we do not employ other LLMs, as is consistent with foundational multi-agent frameworks such as AutoGen [**?**] and MetaGPT [**?**].

Similarly, OR-Tools is employed as the solver, an open-source optimization engine widely used for MIP/LP prototyping in academia and industry. The PY-Agent generates solver agnostic Python code; adapting the same prompt template to Gurobi or CPLEX requires only minimal syntactic substitution (e.g., changing the solver creation call), confirming the framework's portability. Thus, validation under different solvers would assess solver characteristics rather than the efficacy of SOP-MAS itself, which lies beyond the scope of this study.

Following the metrics proposed in [**?**], the evaluation was performed utilizing Compile-Time Errors (CER), Run-Time Errors (RER), and Accuracy Rate (AR). CER quantifies the proportion of instances that fail to compile, RER denotes the proportion of compiled codes encountering failures due to logical errors, resource limitations, or runtime exceptions, and AR represents the proportion of codes that successfully satisfy all test cases. Each algorithm was executed five times with $K = 5$. Mean results are summarized in Table **??**.

The findings suggest that the SPM methodology exhibits the lowest accuracy among all evaluated approaches, mainly due to its black-box reasoning which concentrates solely on the final solution, while neglecting the model's intermediate reasoning steps. This deficiency may result in imprecise outputs when tackling complex problems. In contrast, the CoT method, by employing stepwise reasoning, enhances the model's interpretability and display better performance on complex instances, realizing an increase in accuracy by 8.69% over SPM. Both the CoE and OptiMUS methods reasonably capitalize on multi-agent collaboration to address complex optimization problems effectively. Notably, the proposed SOP-MAS method achieves the highest

accuracy among all baseline approaches.

Notably, the SOP-MAS framework achieves an accuracy rate of 86.59 percent, which falls short of perfect automation. This outcome is neither unexpected nor undesirable; rather, it reflects the intrinsic complexity of real world operational modeling. In practice, even seasoned SCM experts seldom produce a fully correct MIP model on the first attempt. Instead, model development routinely proceeds through multiple cycles of refinement, guided by feasibility diagnostics, sensitivity testing, and iterative feedback from domain stakeholders. Consistent with this reality, SOP-MAS is deliberately designed as a human-in-the-loop decision support tool rather than a fully autonomous optimizer. Its primary objective is to accelerate the expert modeling workflow, from weeks to hours, by automating routine tasks such as data parsing, constraint drafting, and code generation, while simultaneously surfacing potential inconsistencies or violations for expert review. In doing so, the framework shifts human effort away from mechanical implementation toward high value activities such as model validation, business logic verification, and strategic refinement.

4.2. Sensitivity Analysis of LLM Parameters

Within the framework of SOP-MAS system, pivotal parameters encompass the maximum collaboration count K of the PM-Agent, alongside the temperatures of the ME-Agent and PY-Agent, denoted as t_{ME} and t_{PY}, respectively. The parameter K exerts influence over the overall depth and breadth of the workflow by modulating the number of agent invocations. Temperature significantly affects the optimization modeling and solution outputs generated by the agents. Specifically, elevated temperatures foster the development of more innovative solutions, while reduced temperatures bolster deterministic reasoning, and in consequence ensuring stability and compliance of the outputs. This trade-off between precision and creativity mandates meticulous calibration of the temperature parameters tailored to specific task scenarios.

In order to systematically evaluate the effects of parameter configurations on the performance of multi-agent collaboration, a three-dimensional sensitivity analysis framework was developed using the benchmark instance set. The exception backtracking mechanism was deactivated (is_backward=False) to eliminate any confounding variables. A grid search was performed over the parameters $K \in [2, 5]$ and $t_{ME}, t_{PY} \in [0.2, 0.4, 0.8, 1.0]$, utilizing AR as

Figure 3: Faceted Contour Plot of Parameter Sensitivity Analysis

the evaluation metric to identify the optimal parameter combination. The results are shown in Figure **??**.

The PM-Agent orchestrates its constituent agents in accordance with the predefined maximum collaboration threshold K throughout the iterative modeling process. This organization consequently influences the comprehensive scope and extent of the workflow. In instances where K is minimal, the system depends on a limited number of agents to facilitate rapid decision-making, which is appropriate for tasks requiring immediate attention. Conversely, an augmentation in K allows for the incorporation of additional domain-knowledge verification steps, rendering it more suitable for highly complex business scenarios. Notably, as K attains the value of 5, there is a notable enhancement in accuracy, indicating that multi-agent collaboration fosters specialization, with each agent concentrating on a distinct domain.

In contrast, an increase in the parameters t_{ME} and t_{PY} results in declines in accuracy. For both the ME-Agent and PY-Agent, elevated temperatures within the context of mathematical modeling and programming are associated with reduced accuracy. This occurs because lower temperatures substantially limit the randomness in generated outputs, thus ensuring logical rigor and syntactical precision. This is critical for tasks requiring precise derivation, such as mathematical formulae, or strict syntax, such as programming tasks, where sustaining accuracy may slightly constrain creativity but effectively diminishes error rates and enhances development efficiency.

4.3. Ablation Study

An ablation study was conducted to assess the contribution of each agent within SOP-MAS and the effectiveness of the collaborative mechanisms. The fundamental operations of SOP-MAS in addressing optimization challenges is rooted in mathematical modeling and the execution of code. Consequently, this study examines system performance through the selective removal of certain agents or restricting the exception-backtracking mechanism. The experimental groups are defined as follows:

1) **Group 1** (a_{ME}, a_{PY})**:** Solely comprises the ME-Agent and PY-Agent, with exception backtracking deactivated, to assess the significance of collaboration among multiple agents.

2) **Group 2** ($a_{PM}, a_{DE}, a_{ME}, a_{PY}, a_{TE}$)**:** Excluding the BA-Agent whilst permitting exception backtracking to examine the role of the BA-Agent.

3) **Group 3** ($a_{PM}, a_{ME}, a_{PY}, a_{TE}, a_{BA}$)**:** The exclusion of the DE-Agent is permitted, facilitating exception backtracking in order to evaluate the significance of structured modeling information.

4) **Group 4** ($a_{PM}, a_{DE}, a_{ME}, a_{PY}, a_{BA}$)**:** The TE-Agent is omitted to facilitate the exploration of interactions between testing and backtracking mechanisms, while exception backtracking is permitted. In this group, the absence of the TE-Agent results in the non-generation of diagnostic reports. However, the exception backtracking mechanism remains active, enabling each agent to perform self-checks on its output to detect probable errors.

5) **Group 5** ($a_{PM}, a_{DE}, a_{ME}, a_{PY}, a_{TE}, a_{BA}$)**:** All six agents are retained, while exception backtracking is disabled, in order to examine the effect of the exception-backtracking mechanism on system performance.

The entirety of SOP-MAS functions as the control group for assessing performance benchmarks. Within this context, the symbol "+" indicates that exception backtracking is activated, whereas "-" denotes its deactivation. The experimental results are presented in Table **??**.

Table 2: Ablation Study

Group No.	CER	RER	AR
Group1	6.27%	9.80%	57.84%
Group2	1.74%	2.55%	82.58%
Group3	1.74%	3.68%	73.17%
Group4	1.44%	2.09%	84.32%
Group5	3.48%	4.53%	69.69%
Control Group	1.39%	2.91%	86.59%

In Group 1, which consists exclusively of $a_{ME}$and a_{PY}, there is a 28.75% reduction in accuracy compared to the control group. This decline highlighting the significance of multi-agent collaboration in SOP-MAS for tackling complex optimization problems. In Group 2, the removal of the BA-Agent result in an accuracy level of 82.58%, indicating that the BA-Agent exerts a minimal influence on overall accuracy. This is attributed to the position



Figure 4: Average Execution Time and Time Share Distribution

of the BA-Agent at the terminal part of the workflow, where its primary function is the interpretation and packaging of previously obtained results. Consequently, its absence does not compromise the accuracy of front-end modeling and problem-solving but affects solely the presentation and clarity of the final output delivered to the user. The experimental metric, Accuracy Rate (AR), inherently fails to account for the BA-Agent's contribution, and this limitation should be clearly indicated. In Group 3, the removal of the DE-Agent results in a 13.42% reduction in accuracy relative to the control group, demonstrating that structured modeling information is instrumental for the ME-Agent in precisely delineating problem parameters. Group 4 indicates that the exclusion of the TE-Agent still results in a relatively high accuracy, as the testing agent is capable of providing detailed error information, and errors can be rectified via the exception-backtracking mechanism. This observation is consistent with the findings in Group 5, where disabling exception backtracking results in a significant decline in accuracy.

4.4. Computational Efficiency Analysis

To address the reviewer's concern regarding the lack of runtime data and quantification of delays, we present a detailed performance analysis of the proposed LLM-MAS framework. This analysis encompasses 25 representative cases that span a range of problem scales, randomly sampled from the total pool of 269 instances. The findings are summarized in what would be designated as Figure **??**, illustrating the roles of each agent, their average task completion times, and the variability thereof, all measured in seconds.

Our experimental results indicate that the PY-Agent experiences the longest average processing time among all agents, taking approximately 22.47 seconds per task with a standard deviation of 2.03 seconds. This extended duration likely reflects the complexity and variability inherent in coding processes. Conversely, the TE-Agent completes tasks in the shortest average time, requiring only about 1.90 seconds per task, with a notably small standard deviation of 0.11 seconds. This indicates a high level of consistency and efficiency in executing testing tasks.

The other agents, including the DE Agent (8.38 ± 0.73 seconds), ME-Agent (9.21 ± 0.82 seconds), and BE-Agent (6.79 ± 0.61 seconds), also demonstrate efficient performance with acceptable variances, contributing to the overall robustness and practical deployability of the SOP-MAS framework.

Aggregating across all stages, the average end-to-end runtime per instance is approximately 48.75 seconds (sum of mean agent times), with an estimated total standard deviation of 2.4 seconds (propagated from individual variances). This sub-minute latency demonstrates that the framework is well-suited for tactical decision-making scenarios in port operations, where planning horizons typically span hours or days rather than real-time milliseconds.

5. Empirical Analysis of ECR in the Northeastern Hinterland of the Liaoning Coastal Port Cluster

This section selects ECR as the empirical testbed not only because it epitomizes the structural inefficiencies pervasive in global container logistics, but more importantly because it represents a high-stakes, data-rich, and operationally nuanced problem that demands both mathematical rigor and business interpretability, precisely the dual requirements our SOP-MAS framework is designed to fulfill. Unlike generic optimization benchmarks, real-world ECR involves intricate trade-offs among transportation cost, multimodal transport, and contractual obligations across multiple stakeholders, making it an ideal domain for evaluating whether AI-driven modeling can transition from theoretical promise to industrial utility. Within this context, the Northeastern Hinterland of the Liaoning Coastal Port Cluster, encompassing major ports such as Dalian, Yingkou, and Dandong, and their inland logistics networks across Liaoning, Jilin, and Heilongjiang provinces, is chosen as the geographic focus for three compelling reasons.

5.1. Case Setup and Data Sources

The empirical analysis examines the coastal port cluster of Liaoning and its hinterland in Northeast China. Dalian Port serves as a hub for the northeastern economic region, which encompasses the three provinces of Liaoning, Jilin, and Heilongjiang, as well as the four eastern leagues of Inner Mongolia. Yingkou Port and Dalian Port compose a dual-core driving structure. Dandong Port functions as the logistics gateway for the eastern part of Northeast

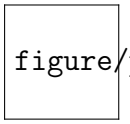


Figure 5: Port Hinterland Map

China. Those three ports share overlapping hinterlands in the inland Northeast, forming key nodes for sea–rail intermodal transportation. Through differentiated strategic positioning, where Dalian Port emphasizing international transshipment, Yingkou Port concentrates on low-cost land transport, and Dandong Port specializes in the eastern border region, the three ports fulfill complementary roles. The port hinterlands are illustrated in Figure **??**.

The examination of empty container repositioning within this regional port cluster entails the involvement of high-dimensional discrete decision variables, constraints specific to the domain, and complex model transformations. This setting serves as an exemplary testbed to evaluate the robustness and generalization capability of the proposed SOP-MAS method in complex, real-world scenarios.

The distances between each port within the Liaoning coastal port cluster and the corresponding dry ports in the Northeastern hinterland are summarized in Table **??**. This investigation adopts a planning horizon of five periods, each with a duration of seven days. Distances between each sea port and dry port are listed in Table **??**, while inter-dry ports are given in Table **??**. As articulated by [**?**] the freight rates applicable to empty container transportation by maritime, road, and rail modalities are determined to be 0.8, 4.5, and 1.8 CNY per TEU per kilometer, respectively. The leasing expense for empty containers at seaports is determined to be 4,000 CNY per TEU with the storage cost for these containers at seaports set at 10 CNY per TEU daily, whereas the average holding cost at dry ports is established at 5 CNY per TEU daily. The supply and demand dynamics of empty containers at each node per period follow a normal distribution, as detailed by [**?**], with specific details provided in Table **??**. Tables **??**, **??**, **??**, and **??** can be found in **??**.

5.2. Problem Description

The ECR network is represented by a directed graph $G = (N, E)$, where the node set N consists of two distinct categories: the seaport set H and

the dry port set L, such that $N = H \cup L$. The collection of edge E symbolizes the routes for the repositioning of empty containers, including pathways between seaports, between dry ports, and between sea- and dry ports. A parameter λ_{ij} is introduced to describe the hinterland affiliation between seaport $i \in H$ and dry port $j \in L$. Specifically, $\lambda_{ij} = 1$ indicates that dry port j belongs to the hinterland of seaport i, thereby permitting the repositioning of empty containers between them. In instances where two dry ports $j_1, j_2 \in L$ are associated with the same seaport $i \in H$ ($\lambda_{ij_1} = \lambda_{ij_2} = 1$), direct repositioning of empty containers between these dry ports is permitted, reflecting the capability for intra-regional collaborative transport. The supply of empty containers primarily originates from the conversion of inbound laden containers. Specifically, laden containers arriving at seaport $i \in H$ in the preceding period are converted into the empty container supply S_i^t for the current period; similarly, laden containers that arrived at dry port $j \in L$ in the preceding period are converted into the empty-container supply S_j^t for the current period. Correspondingly, the outbound laden containers from node $i \in H \cup L$ constitute its empty container demand, D_i^t. Each seaport and dry port node is subject to a maximum storage capacity, $U_i^{\max}$ and incurs a per-period holding cost, c_i^{hold}. When a node experiences an empty container shortage, the demand can be accommodated by repositioning containers from alternative nodes or through the leasing of containers from external providers. The cost associated with repositioning a unit empty container from node i to node j utilizing transport mode $k \in K$ is denoted as c_{ijk}^{repos}, with a maximum transport capacity of $F_{ijk}^{\max}$ for each mode. Leasing a container incurs a cost per unit, represented by c_i^{rent} at node i. In the context of container shipping companies, the issue of repositioning empty container is conceptualized within a two-tier network encompassing both seaports and dry ports over multiple temporal durations. The primary aim is to minimize the aggregate operational expenses, encompassing costs related to repositioning, storage, and leasing. To simplify the model and enhance the ease of its analysis and resolution, the subsequent assumptions have been established:

1) The movement of empty containers may occur among seaports, between seaports and dry ports, as well as among dry ports.

2) Both seaports and dry ports can store empty containers, with the inventory in any period not exceeding the maximum storage capacity.

3) Seaports and dry ports have the capability to lease a virtually unre-

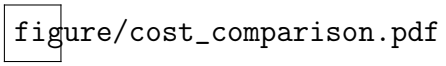


Figure 6: Cost Comparison

stricted quantity of containers during the present operation period.

4) As container shipping services generally adhere to a predetermined weekly schedule, this research employs a 7-day (one-week) decision period as the fundamental unit of planning.

5.3. Results Analysis

Drawing upon the ECR problem described in Section 5.2 and the data presented in Section 5.1, the SOP-MAS framework was employed for end-to-end optimization, with the mathematical model detailed in **??**.

To evaluate the benefits of the sea–land intermodal transport mode, a comparative analysis was conducted under the scenario wherein dry ports are unable to facilitate the exchange of empty containers among themselves, as illustrated in Figure **??**. The results show that the repositioning of empty containers from dispersed dry ports to seaports experiencing shortages becomes increasingly challenging. Consequently, there is an increase in both the volume of empty containers transported directly from dry ports to seaports and the volume circulated among seaports, ultimately leading to elevated overall costs. Furthermore, the exacerbated difficulty in empty container flows leads to a greater number of containers failing to arrive at shortage seaports in a timely manner, which in turn increases the leasing demand at the affected seaports and significantly escalating repositioning costs.

6. Conclusions

In order to address the limitations associated with MIP, which predominantly relies on domain experts and requires problem-specific customization, this study introduces a decision-support framework titled SOP-MAS. This framework integrates large language models with multi-agent theory. The proposed framework facilitates end-to-end optimization by translating natural language descriptions of business optimization problems into actionable business solutions, thereby substantially reducing the application threshold of MIP. To mitigate hallucination issues and the black-box nature of LLM-based

decision making, this study introduces carefully designed prompts, orchestrated workflow based on standard operating procedures, and an exception-backtracking mechanism to enhance interpretability.

The effectiveness of the approach was validated through comparisons with multiple baseline methods on the ComplexOR and NLP4LP benchmark datasets, achieving an accuracy of 86.59%. SOP-MAS was subsequently employed to address the empty container repositioning challenge in the Liaoning coastal port cluster–Northeast hinterland context, leading to a 12.18% reduction in repositioning costs for the shipping company. This application notably curtailed modeling and solution times, expedited prototype development, and furnished scalable decision support amid evolving business rules.

At present, AI agent–based LLMs are incapable of fully supplanting human experts. Domain-specific agents primarily serve as auxiliary copilot AI, emulating the behaviors and decision-making processes of human specialists. Consequently, the proposed method has been experimentally validated solely within the context of MIP and cannot entirely substitute human expertise. As training parameters and scale increase, pre-trained LLMs are anticipated to achieve further advancements in capability. By integrating general-purpose LLMs with customized knowledge bases, the cost associated with decision-making can be significantly reduced. From the perspective of shipping companies, which occupy a central role in the empty container repositioning process, the proposed approach can be further extended to support additional stakeholders, including seaports, dry ports/inland container yards, shippers/customers, and third-party leasing companies.

Code and Data Availability

The complete implementation of the SOP-MAS framework, including all agent prompt templates, benchmark datasets, and empirical case data, is publicly available at `https://github.com/icsco-mako/sop-mas-ecrp`.

Appendix A. Agent Prompt Template

Due to five agents in this work, it is not feasible to present all prompt templates individually. Therefore, we demonstrate the prompt template for the core agent (ME-Agent) as a representative example. The complete prompt templates for all other agents are available in the code repository at `https://github.com/icsco-mako/sop-mas-ecrp`. The following prompt template is designed for the ME-Agent within the SOP-MAS framework:

```
# ROLE: Modeling Expert

## TASK

Review the problem description **{problem_description}** and expert
comments **{message_pool}**. Build a ECR complete mathematical
    model.

## REQUIREMENTS

* Identify and define all decision variables (Continuous / Integer /
  Binary).

* Formulate the objective function with explicit units and check
  dimensional consistency.

* Construct and categorize all constraints (Resource / Logical /
  Business), listing non-negativity constraints separately.

* Do **not** introduce new parameters; only new variables.

* If no new variables are needed, write `[]`.

* Use LaTeX for all math.

* No code required

## OUTPUT (Markdown):

# parameters

(list all parameters)
```

```
# variables

(list all variables)

# objective

(list the objective function)

# constraints

(list all constraints)

# ambiguities

(list any ambiguities)
```

Appendix B. Empirical Case Data

Table B.3: Sea ports repositioning cost and transit time

Sea Port	Sea Port	Distance	Unit Price	Reposition Cost	Period
Dalian Port	Yingkou Port	194.1 km	¥0.8/km	¥155.28	1
Yingkou Port	Dandong Port	340.4 km	¥0.8/km	¥272.32	1
Dandong Port	Dalian Port	315 km	¥0.8/km	¥252	1
Dalian Port	Dandong Port	534.5 km	¥0.8/km	¥427.6	2
Yingkou Port	Dalian Port	655.4 km	¥0.8/km	¥524.32	2
Dandong Port	Yingkou Port	509.1 km	¥0.8/km	¥407.28	2

Table B.4: Port-hinterland distance matrix (km)

Sea Port	Shenyang	Anshan	Harbin	Changchun	Tonghua	Jilin	Tongliao	Chifeng	Daqing	Qiqihar
Dalian Port	379.2	298.8	1015.8	676.1	623	520	460	580	870	930
Yingkou Port	180.4	102.1	747.7	478.6	439.4	540	380	620	-	-
Dandong Port	243.2	236	-	-	275	-	-	-	-	-

Table B.5: Inter-dry port distance matrix (km)

Dry port	Shenyang	Anshan	Harbin	Changchun	Tonghua	Jilin	Tongliao	Chifeng	Daqing	Qiqihar
Shenyang	0	85	-	278	208	343	224	-	-	-
Anshan	85	0	-	-	256	-	-	-	-	-
Harbin	-	-	0	232	-	211	-	-	154	-
Changchun	278	-	232	0	-	100	247	-	301	-
Tonghua	208	256	-	-	0	-	-	-	-	-
Jilin	343	-	211	100	-	0	-	-	327	-
Tongliao	224	285	-	247	-	-	0	311	-	-
Chifeng	376	361	-	-	-	-	311	0	-	-
Daqing	-	-	154	301	-	327	-	-	0	119
Qiqihar	-	-	-	400	-	440	-	-	119	0

Table B.6: Empty container demand and supply at each node (TEU)

Port/EC volume (TEU)	EC demand	EC supply
Shenyang	$(5000, 100^2)$	$(4150, 100^2)$
Anshan	$(1250, 50^2)$	$(1350, 50^2)$
Harbin	$(800, 40^2)$	$(800, 40^2)$
Changchun	$(500, 20^2)$	$(500, 20^2)$
Tonghua	$(500, 20^2)$	$(500, 20^2)$
Jilin	$(500, 20^2)$	$(500, 20^2)$
Tongliao	$(500, 20^2)$	$(500, 20^2)$
Chifeng	$(100, 20^2)$	$(100, 20^2)$
Daqing	$(300, 20^2)$	$(300, 20^2)$
Qiqihar	$(100, 20^2)$	$(100, 20^2)$

Appendix C. ECR Mathematical Model (LaTex)

Since one of our core contributions is the automatic synthesis of publication ready modeling code, we included raw LaTeX snippets primarily to illustrate the output format generated by the SOP-MAS framework.

```
### **Parameters**

- $ H $: Set of seaports
- $ L $: Set of inland ports
- $ N = H \cup L $: Set of nodes
- $ K $: Set of transportation modes (e.g., ship, truck, train)
- $ T $: Set of time periods ($ T = \{1, 2, \ldots, |T|\} $)
- $ \lambda_{ij} \in \{0,1\} $: Hinterland affiliation parameter ($
    i \in H, j \in L $); $ \lambda_{ij} = 1 $ if and only if inland
    port $ j $ belongs to the hinterland of seaport $ i $
- $ \alpha_{j_1 j_2} \in \{0,1\} $: Inland port collaboration
    parameter ($ j_1, j_2 \in L $); $ \alpha_{j_1 j_2} = \max_{h \in
    H} (\lambda_{h j_1} \lambda_{h j_2}) $
- $ S_i^t \geq 0 $: Empty container supply at seaport $ i \in H $
    in period $ t $ (converted from laden containers in the previous
    period)
- $ S_j^t \geq 0 $: Empty container supply at inland port $ j \in L
    $ in period $ t $ (converted from laden containers in the
    previous period)
- $ D_i^t \geq 0 $: Empty container demand at node $ i \in N $ in
    period $ t $ (equal to the volume of laden containers dispatched
    in this period)
- $ U_i^{\max} > 0 $: Maximum storage capacity at node $ i \in N $
- $ c_i^{\text{hold}} > 0 $: Unit inventory holding cost at node $
    i \in N $
- $ c_{ijk}^{\text{repos}} > 0 $: Unit repositioning cost for
    moving an empty container from $ i $ to $ j $ via mode $ k $
- $ F_{ijk}^{\max} \geq 0 $: Maximum transportation capacity on arc
    $ (i,j) $ using mode $ k $
- $ c_i^{\text{rent}} > 0 $: Unit container rental cost at node $ i
    \in N $
- $ \tau_{ij} \geq 1 $: Repositioning lead time (in periods) for
    shipments from seaport $ i $ to seaport $ j $ (defined only for
    $ i,j \in H $)
- $ w_i^0 \geq 0 $: Initial inventory at node $ i $ at the end of
```

```
    period 0 (i.e., beginning of period 1)
- $ M = \sum_{t \in T} \sum_{i \in N} D_i^t $: A sufficiently large
    constant

---

### **Decision Variables**

- $ w_i^t \geq 0 $: Inventory of empty containers at node $ i \in N
    $ at the end of period $ t $ (continuous)
- $ x_{ijk}^t \geq 0 $: Quantity of empty containers repositioned
    from node $ i $ to node $ j $ via mode $ k $ in period $ t $
    (continuous)
- $ y_i^t \geq 0 $: Quantity of containers rented at node $ i \in N
    $ in period $ t $ (continuous)

> **Note**: Auxiliary variable $ z_{ij}^t $ has been removed
    (unused).

---

### **Objective Function**

Minimize total operational cost (repositioning + holding + rental):
$$
\min \sum_{t \in T} \left(
\sum_{i \in N} \sum_{j \in N} \sum_{k \in K} c_{ijk}^{\text{repos}}
    x_{ijk}^t
+ \sum_{i \in N} c_i^{\text{hold}} w_i^t
+ \sum_{i \in N} c_i^{\text{rent}} y_i^t
\right)
$$

---

### **Constraints**

#### **1. Flow Balance Constraints**

**For seaport nodes $ i \in H $** (distinguishing inbound/outbound
```

```
    flows by origin type, incorporating hinterland affiliation and
    lead time):
$$
w_i^{t-1} + S_i^t + y_i^t + \sum_{\substack{j \in H \\ k \in K \\ t
    > \tau_{ji}}} x_{jik}^{t - \tau_{ji}} + \sum_{\substack{j \in L
    \\ k \in K \\ \lambda_{ij} = 1}} x_{jik}^{t} = D_i^t +
    \sum_{\substack{j \in H \\ k \in K}} x_{ijk}^{t} +
    \sum_{\substack{j \in L \\ k \in K \\ \lambda_{ij} = 1}}
    x_{ijk}^{t} + w_i^{t} \quad \forall t \in T
$$

**For inland port nodes $ j \in L $** (strict hinterland
    restriction; all repositioning is instantaneous):
$$
w_j^{t-1} + S_j^t + y_j^t + \sum_{\substack{i \in H \\ k \in K \\
    \lambda_{ij} = 1}} x_{ijk}^{t} + \sum_{\substack{i \in L \\ k
    \in K \\ \alpha_{ij} = 1}} x_{ijk}^{t} = D_j^t +
    \sum_{\substack{i \in H \\ k \in K \\ \lambda_{ij} = 1}}
    x_{jik}^{t} + \sum_{\substack{i \in L \\ k \in K \\ \alpha_{ij}
    = 1}} x_{jik}^{t} + w_j^{t} \quad \forall t \in T
$$

---

#### **2. Hinterland Logic Constraints**

**Seaport and inland port repositioning** (forbidden outside
    designated hinterlands):
$$
x_{ijk}^t = 0 \quad \forall i \in H,  j \in L,  k \in K,  t \in T,
    \text{ if } \lambda_{ij} = 0
$$

$$
x_{jik}^t = 0 \quad \forall i \in H,  j \in L,  k \in K,  t \in T,
    \text{ if } \lambda_{ij} = 0
$$

**Inland port and inland port repositioning** (permitted only if
    ports share a common seaport hinterland):
```

```
$$
x_{j_1 j_2 k}^t \leq M \cdot \alpha_{j_1 j_2} \quad \forall j_1,
    j_2 \in L,  k \in K,  t \in T
$$

> **Remark**:
>
> - $ M = \sum_{t \in T} \sum_{i \in N} D_i^t $ (a sufficiently
    large constant)

---

#### **3. Physical Limit Constraints**

**Transportation capacity constraints** (for all arcs):
$$
x_{ijk}^t \leq F_{ijk}^{\max} \quad \forall i,j \in N,  k \in K,  t
    \in T
$$

**Storage capacity constraints** (for all nodes):
$$
w_i^t \leq U_i^{\max} \quad \forall i \in N,  t \in T
$$

**Non-negativity constraints** (for all variables):
$$
w_i^t \geq 0, \quad x_{ijk}^t \geq 0, \quad y_i^t \geq 0 \quad
    \forall i,j \in N,  k \in K,  t \in T
$$
```

Example citation, See [**?**].

References

[1] B. Li, K. Mellou, B. Zhang, J. Pathuri, I. Menache, Large Language Models for Supply Chain Optimization (Jul. 2023). arXiv:2307.03875, doi:10.48550/arXiv.2307.03875.

[2] P. Li, Q. Yang, S. Xu, X. Li, Z. Li, C. Wang, Y. Liu, T. Guo, J. Tang, Y. Wen, Adaptive-TOD: An LLM-driven and adaptive agent for diverse interaction modes, Neurocomputing 652 (2025) 130991. doi:10.1016/j.neucom.2025.130991.

[3] X. Jiang, Y. Wu, C. Zhang, Y. Zhang, DROC: ELEVATING LARGE LANGUAGE MODELS FOR COMPLEX VEHICLE ROUTING VIA DECOMPOSED RE- TRIEVAL OF CONSTRAINTS (2025).

[4] C. Liu, K. H. Hettige, Q. Xu, C. Long, S. Xiang, G. Cong, Z. Li, R. Zhao, ST-LLM+: Graph Enhanced Spatio-Temporal Large Language Models for Traffic Prediction, IEEE Transactions on Knowledge and Data Engineering 37 (8) (2025) 4846–4859. doi:10.1109/TKDE.2025.3570705.

[5] Y. Ishibashi, Y. Nishimura, Self-Organized Agents: A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization (Apr. 2024). arXiv:2404.02183, doi:10.48550/arXiv.2404.02183.

[6] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions, ACM Transactions on Information Systems 43 (2) (2025) 1–55. doi:10.1145/3703155.

[7] P. Laban, H. Hayashi, Y. Zhou, J. Neville, LLMs Get Lost In Multi-Turn Conversation (May 2025). arXiv:2505.06120, doi:10.48550/arXiv.2505.06120.

[8] D. Zhang, Z. Xiao, Y. Wang, M. Song, G. Chen, Neural TSP Solver with Progressive Distillation, in: B. Williams, Y. Chen, J. Neville (Eds.), Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023,

Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023, AAAI Press, 2023, pp. 12147–12154. doi:10.1609/AAAI.V37I10.26432.

[9] R. Ramamonjison, T. Yu, R. Li, H. Li, G. Carenini, B. Ghaddar, S. He, M. Mostajabdaveh, A. Banitalebi-Dehkordi, Z. Zhou, Y. Zhang, NL4Opt Competition: Formulating Optimization Problems Based on Their Natural Language Descriptions, in: Proceedings of the NeurIPS 2022 Competitions Track, PMLR, 2023, pp. 189–203.

[10] X. Xiang, X. Xu, C. Liu, S. Jia, Liner fleet deployment and empty container repositioning under demand uncertainty: A robust optimization approach, Transportation Research Part B: Methodological 190 (2024) 103088. doi:10.1016/j.trb.2024.103088.

[11] X. Xiang, C. Liu, S. Jia, Tactical vessel deployment and empty container repositioning considering container turnover times, Computers & Industrial Engineering 190 (2024) 110032. doi:10.1016/j.cie.2024.110032.

[12] X. Xu, X. Huang, L. Wang, Empty container repositioning problem using a reinforcement learning framework with multi-weight adaptive reward function, Maritime Policy & Management (2024) 1–22doi:10.1080/03088839.2024.2326635.

[13] K. A. Kuźmicz, E. Pesch, Container depot location problem in the frame of the Polish part of the New Silk Road, MATEC Web of Conferences 312 (2020) 02004. doi:10.1051/matecconf/202031202004.

[14] W. Shan, Z. Peng, J. Liu, B. Yao, B. Yu, An exact algorithm for inland container transportation network design, Transportation Research Part B: Methodological 135 (2020) 41–82. doi:10.1016/j.trb.2020.02.011.

[15] H. Zhang, L. Lu, X. Wang, Tactical and Operational Cooperative Empty Container Repositioning Optimization Model Based on Business Flow and Initial Solutions Generation Rules, Symmetry 11 (3) (2019) 300. doi:10.3390/sym11030300.

[16] J. Liang, Z. Ma, S. Wang, H. Liu, Z. Tan, Dynamic container slot allocation with empty container repositioning under stochastic demand, Transportation Research Part E: Logistics and Transportation Review 187 (2024) 103603. doi:10.1016/j.tre.2024.103603.

[17] M. Yu, X. Tan, J. Chen, Pricing competition in maritime transportation with blockchain technology and empty container repositioning, Advanced Engineering Informatics 59 (2024) 102318. doi:10.1016/j.aei.2023.102318.

[18] S. Fazi, S. K. Choudhary, J.-X. Dong, The multi-trip container drayage problem with synchronization for efficient empty containers re-usage, European Journal of Operational Research 310 (1) (2023) 343–359. doi:10.1016/j.ejor.2023.02.041.

[19] J. Cai, Y. Huang, C. Diao, Z. Jin, Joint optimization of empty container repositioning and inventory control applying dynamic programming and simulated annealing, Applied Soft Computing 167 (2024) 112452. doi:10.1016/j.asoc.2024.112452.

[20] T. Luo, D. Chang, Empty container repositioning strategy in intermodal transport with demand switching, Advanced Engineering Informatics 40 (2019) 1–13. doi:10.1016/j.aei.2019.02.008.

[21] Y. Xie, X. Liang, L. Ma, H. Yan, Empty container management and coordination in intermodal transport, European Journal of Operational Research 257 (1) (2017) 223–232. doi:10.1016/j.ejor.2016.07.053.